

WEB SİTESİ NASIL ÇALIŞIR

[Kaynak](#)

İÇERİK

1. How Websites Work (Web Siteleri Nasıl Çalışır?)
2. HTML: Web'in İskeleti ve Yapı Taşları
3. JavaScript: Web'in Dinamik Beyni ve Etkileşim Gücü
4. Sensitive Data Exposure (Hassas Veri İfşası)
5. HTML Injection (HTML Enjeksiyonu)

How Websites Work (Web Siteleri Nasıl Çalışır?)

Bu ders notlarında, bir web sitesine tıkladığımız andan ekrana görüntü gelene kadar arka planda dönen süreçleri, istemci ve sunucu arasındaki o kritik diyalogu ve güvenliğin ilk temel taşlarını inceleyeceğiz. Hedefimiz, web'in mimarisini bir siber güvenlik perspektifiyle anlamak.

Web İletişiminin Temel Mantığı: İstek ve Yanıt (Request & Response)

Bir web sitesini ziyaret ettiğinizde (örneğin Google Chrome veya Safari kullanarak), aslında bir **istek (request)** başlatmış olursunuz. Bu süreç, sizin tarayıcınız ile dünyanın herhangi bir yerinde bulunan ve **Web Sunucusu (Web Server)** olarak adlandırılan özel bir bilgisayar arasında gerçekleşir.

- **Süreç:** Tarayıcınız sunucuya "Bana bu sayfanın bilgilerini gönder" der.
- **Yanıt:** Sunucu bu isteği işler ve tarayıcınızın sayfayı size görsel olarak sunabilmesi için gerekli verileri içeren bir **yanıt (response)** gönderir.
- **Web Sunucusu Nedir?** Sadece sizin isteklerinizi karşılamak için tasarlanmış, internete bağlı, yüksek performanslı ve "dinleyen" bir bilgisayardır.

Web Sitelerinin İki Ana Bileşeni

Bir web sitesini sadece ekranda gördüğümüz renkli kutucuklar olarak düşünemeyiz. Siber güvenlikte bir açığı ararken, hatanın "istemci" tarafında mı yoksa "sunucu" tarafında mı olduğunu bilmemiz gerekir.

1. Front End (Client-Side - İstemci Tarafı)

Tarayıcınızın bir web sitesini görselleştirmek (render etmek) için kullandığı kısımdır. Kullanıcı olarak sizin doğrudan etkileşime girdiğiniz, butonlara bastığınız ve yazıları okuduğunuz yer

burasıdır.

- **Güvenlik Notu:** İstemci tarafında gerçekleşen işlemler (JavaScript gibi), kullanıcı kontrolüne açıktır. Dolayısıyla buradaki kısıtlamalar genellikle "aşılabilir" kabul edilir.

2. Back End (Server-Side - Sunucu Tarafı)

İsteğinizi işleyen, veritabanına sorgu atan ve sonucu size paketleyip gönderen "mutfak" kısmıdır.

- **İşleyiş:** Siz bir butona bastığınızda, bu istek sunucuya gider. Sunucu arkada gerekli hesaplamaları yapar veya verileri kontrol eder, ardından sonucu size döner.
- **Güvenlik Notu:** Asıl kritik açıklar (SQL Injection, RCE vb.) genellikle sunucu tarafındaki mantık hatalarından veya yetersiz girdi denetiminden kaynaklanır.

Özet Akış Şeması

Bu yapıyı aylar sonra hatırlamak için şu basit zinciri takip etmeliyiz:

1. **Tarayıcı (Client):** "Hey Sunucu, bana index.html dosyasını ver!" (**HTTP Request**)
2. **Sunucu (Server):** "İsteğini aldım, yetkilerini kontrol ettim, işte dosya içeriği." (**HTTP Response**)
3. **Tarayıcı (Render):** Sunucudan gelen ham veriyi (HTML, CSS, JS) alıp bizim görebileceğimiz görsel bir arayüze dönüştürür.

Neden Önemli? Bir web hacking senaryosunda, araya girip bu istekleri değiştirebiliriz (Interception). Eğer sunucu tarafındaki doğrulamalar zayıfsa, sunucuyu normalde yapmaması gereken bir işi yapmaya zorlayabiliriz.

HTML: Web'in İskeleti ve Yapı Taşları

Bir web sitesinin nasıl görüldüğünden ziyade, siber güvenlikçiler olarak bizim için **nasıl inşa edildiği** daha kritiktir. Bir web sitesi temel olarak üç ana teknolojinin birleşimiyle oluşur:

- **HTML:** Web sitesinin yapısını ve içeriğini belirleyen iskelettir.
- **CSS:** Bu iskelete görsel estetik (renk, düzen, yazı tipi) kazandıran makyajdır.
- **JavaScript:** Sayfaya etkileşim ve karmaşık özellikler (buton tıklamaları, veri doğrulama, animasyonlar) ekleyen beyin fonksiyonlarıdır.

HTML Nedir? (HyperText Markup Language)

HTML, web sitelerinin yazıldığı dildir. Ancak bu bir programlama dili değil, bir **işaretleme (markup)** dilidir. Tarayıcıya, hangi metnin bir başlık, hangisinin bir paragraf veya hangisinin bir resim olduğunu "etiketler" kullanarak söyler.

Temel HTML Yapısı

İstisnasız her standart web sitesi şu temel hiyerarşiyi takip eder:

HTML

```
<!DOCTYPE html>
<html>
  <head>
    <title>Sayfa Başlığı</title>
  </head>
  <body>
    <h1>Bu Büyük Bir Başlıktır</h1>
    <p>Bu bir paragraf örneğidir.</p>
  </body>
</html>
```

Bileşenlerin Detaylı Analizi:

- `<!DOCTYPE html>` : Tarayıcıya bu belgenin bir **HTML5** belgesi olduğunu bildirir. Farklı tarayıcıların sayfayı aynı standartta yorumlamasını sağlar. Eğer bu olmazsa tarayıcı "quirks mode" (tuhaflik modu) dediğimiz eski bir yöntemle geçebilir ve sayfa hatalı görünebilir.
- `<html>` : Sayfanın kök (**root**) elementidir. Diğer tüm HTML kodları bu etiketin içinde yaşar.
- `<head>` : Sayfa hakkındaki meta bilgileri (sayfa başlığı, karakter seti, CSS dosyaları) içerir. Buradaki içerikler tarayıcı penceresinde doğrudan kullanıcıya **görünmez**.
- `<body>` : Web sayfasının ana gövdesidir. Tarayıcı ekranında gördüğümüz her şey (yazılar, resimler, butonlar) bu etiketin içindedir.
- `<h1>` ile `<h6>` **arası**: Başlıkları tanımlar. `<h1>` en büyük ve en önemli başlıktır.
- `<p>` : Paragrafları (Paragraph) tanımlar.

Etiketler ve Öznitelikler (Attributes)

HTML etiketleri sadece metni sarmalamakla kalmaz, aynı zamanda **öznitelikler (attributes)** aracılığıyla ek bilgiler de taşırlar. Bu öznitelikler siber güvenlikte "input" (girdi) noktalarını bulmak için hayati önem taşır.

Örnek bir yapı: `<p attribute1="value1" attribute2="value2">İçerik</p>`

Kritik Öznitelik Türleri:

1. **Sınıf (class)**: Birden fazla elemente aynı stili (CSS) uygulamak için kullanılır.
 - Örnek: `<p class="error-msg">Hata!</p>` (Tüm hata mesajları kırmızı olsun gibi).
2. **ID (id)**: Bir elemente özel, **benzersiz (unique)** bir kimlik tanımlar. Bir sayfada aynı ID'ye sahip sadece bir element olmalıdır. JavaScript bir elemente müdahale edeceği zaman genellikle bu ID'yi kullanır.

- Örnek: `<p id="username-display">Admin</p>`

3. **Kaynak (src)**: Genellikle resimler (`<img>`) veya scriptler için kullanılır, dosyanın nerede olduğunu belirtir.

- Örnek: ``
- **Güvenlik Notu**: Eğer bir saldırgan `src` içeriğini manipüle edebilirse, sayfaya zararlı bir script (XSS) enjekte edebilir.

Bir Siber Güvenlikçinin Gözünden HTML

Bizim için HTML sadece metin değildir; o bir **saldırı yüzeyidir**. Bir web sitesinin HTML kodunu incelemek (Analiz), zafiyet avcılığının (Enumeration) ilk adımıdır.

- **View Page Source (Sayfa Kaynağını Görüntüle)**: Tarayıcıda sağ tıklayıp bu seçeneği seçtiğinizde, sunucunun size gönderdiği ham HTML kodunu görürsünüz.
- **Neden Bakarız?**
 - Geliştiricilerin unuttuğu gizli yorum satırlarını (``) bulmak.
 - Gizli form alanlarını (`<input type="hidden">`) tespit etmek.
 - Kullanılan framework'leri veya admin paneli yollarını deşifre etmek.

Unutma: Tarayıcıda gördüğün her şey sunucunun sana "güvenip" gönderdiği verilerdir. HTML üzerinde yapacağın inceleme, arka planda çalışan mantığı anlamamanın en hızlı yoludur.

JavaScript: Web'in Dinamik Beyni ve Etkileşim Gücü

HTML sitenin iskeletini oluştururken, **JavaScript (JS)** bu iskelete hayat veren, onu hareket ettiren ve kullanıcıyla iletişim kurmasını sağlayan bileşendir. JS olmadan web sayfaları sadece okunabilir sabit dökümanlar (statik) olurdu; ancak JS sayesinde sayfalar, biz sayfayı yenilemeden bile anlık olarak güncellenebilir (dinamik).

JavaScript Nedir ve Ne İşe Yarar?

JavaScript, dünyadaki en popüler kodlama dillerinden biridir ve birincil görevi web sayfalarına **işlevsellik** katmaktır.

- **Dinamik Güncelleme**: Sayfayı kapatıp açmaya gerek kalmadan içeriği değiştirir.
- **Olay Kontrolü (Event Handling)**: Bir butona tıklandığında, fare üzerine getirildiğinde veya klavyeden bir tuşa basıldığında ne olacağını belirler.
- **Görsel Efektler**: Hareketli animasyonlar, açılır pencereler ve stil değişiklikleri JS ile yönetilir.

JavaScript Sayfaya Nasıl Eklenir?

Bir geliştirici, JavaScript kodlarını sayfaya iki farklı yöntemle dahil edebilir. Siber güvenlik testlerinde (Pentest) her iki yöntemi de kontrol etmek kritik öneme sahiptir:

1. **Satır İçi (Inline) Kullanım:** Doğrudan HTML dosyası içinde `<script>` etiketleri arasına yazılır.

HTML

```
<script>
  console.log("Bu kod doğrudan HTML içinden çalışıyor.");
</script>
```

2. **Harici (External) Kullanım:** Kodlar başka bir dosyada tutulur ve `src` özneliği ile sayfaya çağrılır.

HTML

```
<script src="/js/ana_dosya.js"></script>
```

Güvenlik Notu: Harici dosyaların yollarını incelemek, bazen sunucuda unutulmuş yedek JS dosyalarını veya hassas bilgi sızdıran scriptleri bulmamızı sağlar.

Sayfa İçeriğini Manipüle Etmek (DOM Manipülasyonu)

JavaScript, HTML elementlerine erişebilir ve onları değiştirebilir. Bunun en temel örneği, bir elementin **ID**'sini kullanarak içeriğini değiştirmektir.

Örnek Kod:

JavaScript

```
document.getElementById("demo").innerHTML = "Hack the Planet";
```

- **document** : Mevcut web sayfasını temsil eden kök nesne.
- **.getElementById("demo")** : Sayfa içinde `id="demo"` olan spesifik elementi bulur.
- **.innerHTML** : O elementin içindeki HTML içeriğine (yazıya) erişir.
- **Hack the Planet**: İçeriği bu yeni metinle değiştirir.

Olaylar (Events) ve Tetikleyiciler

HTML elementleri, kullanıcı bir aksiyon aldığında JavaScript çalıştırmak üzere tasarlanmış **olay (event)** özneliklerine sahip olabilir.

1. **onclick** (Tıklama Olayı)

Kullanıcı bir butona veya elemente tıkladığında çalışır.

HTML

```
<button onclick='document.getElementById("demo").innerHTML = "Buton  
Tıklandı!";'>Tıkla Beni!</button>
```

Bu örnekte, butona basıldığı an JavaScript devreye girer ve "demo" ID'li yerin metnini değiştirir.

2. onhover (Üzerine Gelme Olayı)

Fare imleci bir elementin üzerine geldiğinde tetiklenir.

Önemli Not: Bu olaylar sadece elementlerin içinde değil, doğrudan script dosyaları içinde de tanımlanabilir. Bu, kodun daha temiz tutulmasını sağlar ancak bir güvenlikçi için "bu buton tıklandığında ne çalışıyor?" sorusunun cevabını aramayı zorlaştırabilir.

Bir Siber Güvenlikçinin Gözünden JavaScript

JavaScript, siber güvenlik dünyasında en çok **XSS (Cross-Site Scripting)** zafiyeti ile anılır.

- **Zayıf Nokta:** Eğer bir web sitesi, kullanıcıdan aldığı girdiyi (örneğin bir yorum kutusuna yazılan metni) hiçbir filtreden geçirmeden sayfaya basıyorsa, saldırgan oraya bir `<script>` etiketi yazabilir.
- **Sonuç:** Diğer kullanıcılar o sayfayı açtığında, saldırganın yazdığı JavaScript kodları o kullanıcıların tarayıcısında çalışır. Bu da session (oturum) çalma veya sayfayı manipüle etme gibi ciddi sonuçlar doğurur.

Sensitive Data Exposure (Hassas Veri Ifşası)

Bir web uygulamasında güvenlik analizi yaparken ilk bakılması gereken noktalardan biri, geliştiricinin "nasıl olsa kimse buraya bakmaz" diyerek bıraktığı izlerdir. **Sensitive Data Exposure**, bir web sitesinin son kullanıcıya ulaştırmaması gereken hassas bilgileri (açık metin şifreler, gizli yollar, konfigürasyon detayları vb.) koruyamaması veya temizlememesi durumunda ortaya çıkar. Bu sızıntılar genellikle uygulamanın **Frontend** (ön yüz) kaynak kodunda saklı kalır.

Kaynak Kodun İncelenmesi ve Temel Mantık

Web siteleri birçok HTML elementinden (tag) oluşur ve tarayıcımız bu sayfayı işlerken tüm bu kodları indirir. Biz de "View Page Source" (Sayfa Kaynağını Görüntüle) diyerek bu ham yapıya ulaşabiliriz. Geliştiriciler bazen geliştirme aşamasında kolaylık olması için kodun içine notlar bırakır veya test hesaplarını orada unuttur.

1. HTML Yorum Satırları (HTML Comments)

Geliştiriciler, kodun ne işe yaradığını hatırlamak için `` etiketlerini kullanırlar. Ancak bu etiketler sunucu tarafında değil, **istemci tarafında** (bizim tarayıcımızda) yorumlanır.

- **Tehlike:** Bu yorumlar içinde geçici giriş bilgileri (**temporary credentials**), veritabanı bağlantı dizileri veya admin paneline giden gizli linkler bulunabilir.
- **Saldırı Vektörü:** Eğer kaynak kodda bir kullanıcı adı ve şifre bulursanız, bu bilgileri uygulamanın giriş panellerinde veya varsa SSH/FTP gibi arka uç (backend) bileşenlerinde deneyebilirsiniz.

2. JavaScript Dosyaları ve Gizli API Key'ler

Sadece HTML değil, sayfaya dahil edilen `.js` dosyaları da altın değerindedir. Modern web uygulamaları API'lar ile konuşur.

- **Neye Bakmalı?:** JavaScript dosyaları içinde hard-coded (kodun içine gömülmüş) halde bulunan **API Keyler**, AWS Access Keyler veya gizli endpoint (uç nokta) adresleri, saldırganın uygulama yetkilerini aşmasına ve doğrudan sunucu servislerine erişmesine neden olabilir.

Uygulama Adımları: Nereden Başlamalıyız?

Bir sızma testi veya CTF (Capture The Flag) sürecinde bu zafiyeti sömürmek için şu izlek takip edilmelidir:

1. **Sayfa Kaynağını Aç:** Tarayıcıda sağ tıklayıp `Sayfa Kaynağını Görüntüle` (CTRL+U) komutunu verin.
2. **Arama Yap (CTRL+F):** Kodun içinde manuel kaybolmak yerine stratejik anahtar kelimeleri aratın:
 - `pass`, `password`, `user`, `admin` (Giriş bilgileri için)
 - `config`, `key`, `secret` (Konfigürasyon sızıntıları için)
 - `hidden`, `api`, `dev`, `test` (Gizli yollar veya test ortamları için)
3. **Yorum Satırlarını Ayıkla (HTML Comments)**
 - Yarım kalan yerden devam edelim: `` gibi notlar bırakır. Bu notlar bize sistemin henüz tamamlanmamış, zayıf noktalarını fısıldar.
 - **Kritik Detay:** Eğer bir yorumda `` gibi bir dizin görürseniz, bu sayfa muhtemelen **Robots.txt** dosyasında bile yoktur. Bu, doğrudan "Gizli Dizin Keşfi" (**Directory Discovery**) anlamına gelir.
4. **Harici Dosyaları ve Kütüphaneleri Takip Et**

Kaynak kodda sadece HTML metnine bakmak yetmez. Sayfanın çağırdığı tüm dış dosyalar incelenmelidir.

 - **CSS Dosyaları (`.css`):** Nadiren de olsa, CSS dosyalarının en alt kısmına yorum satırı olarak geliştirici notları veya sürüm bilgileri eklenebilir.
 - **Manifest Dosyaları (`.json`, `.xml`):** PWA (Progressive Web Apps) gibi yapılarda `manifest.json` dosyası, uygulamanın mimarisi hakkında ipuçları verir.

HTML Injection (HTML Enjeksiyonu)

Bu zafiyet, kullanıcıdan alınan verilerin hiçbir filtreden geçirilmeden (**unfiltered**) doğrudan sayfa üzerinde görüntülenmesi sonucu oluşur. Eğer bir web sitesi "input sanitisation" (girdi temizleme) işlemini yapmazsa, saldırgan web sayfasına kendi HTML kodlarını enjekte edebilir.

Temel Mantık: Girdi Temizleme (Sanitisation) Nedir?

Güvenli bir web sitesinin altın kuralı şudur: **Kullanıcıdan gelen girdiye asla güvenme!** Kullanıcının bir forma yazdığı metin, genellikle uygulamanın frontend veya backend fonksiyonlarında kullanılır.

- **Sanitisation (Temizleme):** Kullanıcının girdiği verideki tehlikeli karakterlerin (örneğin `<` `>` `/` `"` `'` gibi HTML etiketlerini oluşturan karakterler) ayıklanması veya zararsız hale getirilmesidir.
- **Zafiyetin Oluşumu:** Eğer geliştirici bu temizleme işlemini yapmazsa, tarayıcı bu girdiyi "düz metin" olarak değil, "çalıştırılabilir kod" (HTML/JavaScript) olarak algılar.

Not: Veritabanı enjeksiyonu (SQL Injection) gibi zafiyetler backend'i hedef alırken, HTML Injection **client-side** (istemci tarafı) bir zafiyettir. Yani doğrudan son kullanıcının tarayıcısını etkiler.

Zafiyetin Anatomisi: sayHi Fonksiyonu Örneği

Diyelim ki bir sitede "Adınız nedir?" sorusunu soran bir form var. Formun işleyiş mantığı şu şekildedir:

1. Kullanıcı ismini girer (Örn: Ahmet).
2. Bu girdi bir JavaScript fonksiyonuna (örneğin sayHi) iletilir.
3. Fonksiyon bu ismi alır ve sayfaya `<h1>Merhaba Ahmet</h1>` şeklinde basar.

Saldırı Senaryosu (Exploitation)

Eğer ben ismimi yazmak yerine HTML kodu girersem ne olur?

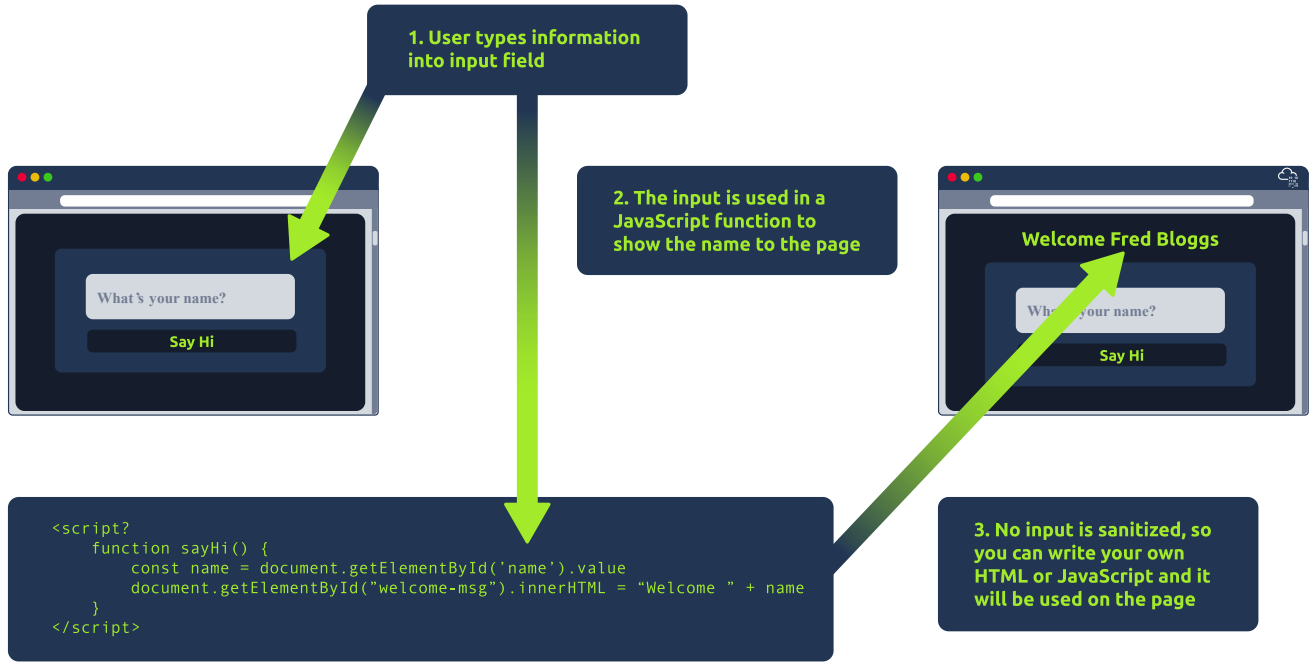
- **Input:** `<u>Not Tut</u>`
- **Sistemin Çıktısı:** `<h1>Merhaba <u>Not Tut</u></h1>`
- **Sonuç:** Sayfada ismim altı çizili görünür. Tarayıcı `<u>` etiketini başarıyla yorumlamıştır.

Daha ileri giderek sayfa yapısını tamamen bozabiliriz:

HTML

```
<marquee><h1>Hacked by Not Tut</h1></marquee>
```

Bu girdi, sayfada kayan devasa bir başlık oluşturacaktır. Bu durum, saldırganın sayfanın görünümünü ve işlevselliğini kontrol edebildiğini kanıtlar (**Defacement**).



HTML Injection Neden Tehlikelidir?

Sadece "sayfada kayan yazı yazmak" masum görünebilir, ancak HTML Injection çok daha ciddi saldırıların kapısını aralar:

1. **Phishing (Oltalama):** Saldırgan, yasal sitenin içine sahte bir giriş formu (login form) enjekte edebilir. Kullanıcı, şifresini asıl siteye girdiğini sanırken aslında saldırganın hazırladığı forma yazar.
2. **Sayfa Yönlendirme:** `<meta http-to-equiv="refresh" ...>` gibi etiketlerle kullanıcıyı zararlı bir siteye yönlendirebilir.
3. **XSS'e Geçiş:** HTML Injection yapılabiliyorsa, genellikle **XSS (Cross-Site Scripting)** de yapılabilir. Yani `<script>` etiketleri kullanarak kullanıcıların **Session Cookie** (oturum çerezleri) bilgilerini çalmak mümkündür.

Korunma Yöntemleri (Remediation)

Geliştiricilerin bu zafiyeti önlemek için alması gereken önlemler basittir:

- **Etiketlerin Kaldırılması:** Kullanıcıdan gelen metindeki tüm `<` ve `>` gibi karakterleri silmek veya filtrelemek.
- **Encoding (Kodlama):** HTML karakterlerini "entity" karşılıklarına dönüştürmek.
 - `<` yerine `<`;
 - `>` yerine `>`;
 - Bu sayede tarayıcı bunu kod olarak değil, ekranda görünecek sembol olarak algılar.

Teknik Özet Tablosu

Durum	Kullanıcı Girdisi	Tarayıcıda Görünen	Sonuç
Güvenli	Test	Test	HTML encode edilmiş, zarar yok.
Zafiyetli	Test	Test (Büyük puntolu)	HTML yorumlanmış, Injection Başarılı!